Quality was a key part of our development process and as such, several steps were taken to increase maintainability and reduce dependencies between components.

This was implemented using object oriented design principles such as encapsulation, polymorphism, singletons and controllers so that the program wouldn't break when changes are made.

Along with this, we ensured the entire codebase was properly documented with inline comments, function descriptions, and javadocs documentation.

As a result, development time was reduced while overall code quality greatly improved.

There were some limits to our program that needed to be overcome, such as limited access to live data which constrained realism, and requiring abstraction in design due to time constraints which we accounted for by adapting the application and keeping strict deadlines.

Within our development process, we learnt key lessons of working in a software team. A lesson was highlighted by the challenge of maintaining consistent quality and design principles across all team members' code. This was crucial to ensure the customer satisfaction was met, ensuring our alignment with the twelve principles of agile.

Deadlines were also demonstrated to be crucial, as it will allow us to do frequent delivery of software to customers. Feedback was another critically evaluated lesson as code quality and development was important, reinforcing Agile's emphasis on iterative improvement and continuous feedback.

Additionally, the professional insights consisted of achieving quality standards of user story development, to ensure features and software achieved the user expectations. Tasks were also delegated effectively, based on team strengths and skills to ensure tasks were completed effectively and on time. This also created integration challenges later in development.

Only 20% of traders remain profitable over time, this highlights that most beginners fail due to lack of education and guidance.

There are currently few educational trading simulators that provide support such as AI-driven feedback or performance analysis.

Stockle addresses this gap by offering a beginner-friendly, paper trading platform that simulates the US stock market.

Using an AI assistant, Stockle provides personalised feedback, and performance analysis. This allows users to learn from simulated decisions without financial risk, helping them learn through immediate assistance.

One of the twelve principles of agile development is "Deliver working software frequently" This was built into the core of our development process through continuous development. We used GitHub as our choice for a repository hosting service over other alternatives like git bucket due to familiarity and popularity of the platform. Github made it seamless to collaborate and maintain a functional prototype to showcase for stakeholders however required constant attention with branching to avoid conflicts.

To further avoid these conflicts Junit tests were implemented through GitHub actions on each pull request with peer review. However this had a large overhead compared to manual testing, future development will greatly benefit from this however.

The entire development process focused on the user experience where at each step of development we asked ourselves what would the user think. For the majority of development, we assumed what end users wanted to save time with user testing, which was considerably successful since each team member had various experience and understanding of the stock market. These user considerations heavily influenced the original concept, UI design and were the reason for using user story based feature development. However this design did limit the ability to work on overall system architecture. To alleviate this issue at the beginning of development large architecture tasks were assigned to individual developers.

6 potential Users were also tested later into development. These tests revealed flaws in our application which led to many user stories being created and multiple being developed. Such as adding tooltips and ability to change timeframe for candlestick graphs. In reflection continuous testing throughout development would have revealed these changes earlier avoiding changing them later.